RequestToken

`class` authx.schema.RequestToken

Bases: `BaseModel`

Verify and validate a token with comprehensive security checks.

Performs multiple layers of token validation including JWT decoding, type verification, CSRF protection, and freshness checks.

PARAMETER	DESCRIPTION
`key`	The cryptographic key used for token verification.
`algorithms`	Optional list of allowed cryptographic algorithms. Defaults to HS256.
`audience`	Optional expected token audience.
`issuer`	Optional expected token issuer.
`verify_jwt`	Flag to enable JWT verification. Defaults to True.
`verify_type`	Flag to validate token type matches expected type. Defaults to True.
`verify_csrf`	Flag to perform Cross-Site Request Forgery protection. Defaults to True.
`verify_fresh`	Flag to require a fresh token. Defaults to False.

RETURNS	DESCRIPTION
	A validated TokenPayload instance representing the decoded token.

RAISES	DESCRIPTION
`JWTDecodeError`	If token decoding fails.
`TokenTypeError`	If token type does not match expected type.
`FreshTokenRequiredError`	If a fresh token is required but not provided.

RAISES	DESCRIPTION
`CSRFError`	If CSRF token validation fails.

attr **token** `class-attribute` `instance-attribute`

```
token = Field(..., description='The token to verify')
```

attr **csrf** `class-attribute` `instance-attribute`

```
csrf = None
```

attr **type** `class-attribute` `instance-attribute`

```
type = 'access'
```

attr **location** `instance-attribute`

```
location
```

meth **verify**

```
verify(key, algorithms=None, audience=None, issuer=None, verify_jwt=True, verify_type=True,
verify_csrf=True, verify_fresh=False)
```

Verify and validate a token with comprehensive security checks.

Performs multiple layers of token validation including JWT decoding, type verification, CSRF protection, and freshness checks.

PARAMETER	DESCRIPTION
`key`	The cryptographic key used for token verification. **TYPE:** `str`
`algorithms`	Optional list of allowed cryptographic algorithms. Defaults to HS256. **TYPE:** `Optional[Sequence[AlgorithmType]]` **DEFAULT:** `None`
`audience`	Optional expected token audience. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`issuer`	Optional expected token issuer. **TYPE:** `Optional[str]` **DEFAULT:** `None`
`verify_jwt`	Flag to enable JWT verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`verify_type`	Flag to validate token type matches expected type. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`

PARAMETER	DESCRIPTION
`verify_csrf`	Flag to perform Cross-Site Request Forgery protection. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`verify_fresh`	Flag to require a fresh token. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`

RETURNS	DESCRIPTION
`TokenPayload`	A validated TokenPayload instance representing the decoded token.

RAISES	DESCRIPTION
`JWTDecodeError`	If token decoding fails.
`TokenTypeError`	If token type does not match expected type.
`FreshTokenRequiredError`	If a fresh token is required but not provided.
`CSRFError`	If CSRF token validation fails.

Source code in `authx/schema.py`

```python
def verify(
    self,
    key: str,
    algorithms: Optional[Sequence[AlgorithmType]] = None,
    audience: Optional[StringOrSequence] = None,
    issuer: Optional[str] = None,
    verify_jwt: bool = True,
    verify_type: bool = True,
    verify_csrf: bool = True,
    verify_fresh: bool = False,
) -> TokenPayload:
    """Verify and validate a token with comprehensive security checks.

    Performs multiple layers of token validation including JWT decoding, type verification, CSRF
protection, and freshness checks.

    Args:
        key: The cryptographic key used for token verification.
        algorithms: Optional list of allowed cryptographic algorithms. Defaults to HS256.
        audience: Optional expected token audience.
        issuer: Optional expected token issuer.
        verify_jwt: Flag to enable JWT verification. Defaults to True.
        verify_type: Flag to validate token type matches expected type. Defaults to True.
        verify_csrf: Flag to perform Cross-Site Request Forgery protection. Defaults to True.
        verify_fresh: Flag to require a fresh token. Defaults to False.

    Returns:
        A validated TokenPayload instance representing the decoded token.

    Raises:
        JWTDecodeError: If token decoding fails.
        TokenTypeError: If token type does not match expected type.
        FreshTokenRequiredError: If a fresh token is required but not provided.
        CSRFError: If CSRF token validation fails.
    """
    if algorithms is None:  # pragma: no cover
        algorithms = ["HS256"]  # pragma: no cover
    try:
        decoded_token = decode_token(
            token=self.token,
            key=key,
            algorithms=algorithms,
            verify=verify_jwt,
            audience=audience,
            issuer=issuer,
        )
        payload = TokenPayload.model_validate(decoded_token)
    except JWTDecodeError as e:
        raise JWTDecodeError(*e.args) from e
    except ValidationError as e:
        raise JWTDecodeError(*e.args) from e

    if verify_type and (self.type != payload.type):
        error_msg = f"'{self.type}' token required, '{payload.type}' token received"
        if self.type == "access":
            raise AccessTokenRequiredError(error_msg)
        elif self.type == "refresh":  # pragma: no cover
            raise RefreshTokenRequiredError(error_msg)  # pragma: no cover
        raise TokenTypeError(error_msg)  # pragma: no cover

    if verify_fresh and not payload.fresh:
        raise FreshTokenRequiredError("Fresh token required")

    if verify_csrf and self.location == "cookies":
        if self.csrf is None:
            raise CSRFError(
                f"Missing CSRF token in request. Include CSRF token from the "
                f"'{self.location}' in the 'X-CSRF-TOKEN' header. "
                f"To disable CSRF protection, set JWT_COOKIE_CSRF_PROTECT=False."
            )
        if payload.csrf is None:
            raise CSRFError("Cookies token missing CSRF claim")  # pragma: no cover
```

```
        if not compare_digest(self.csrf, payload.csrf):
            raise CSRFError("CSRF token mismatch")

    return payload
```

March 20, 2026 March 19, 2026 GitHub